# OS Orange Problem – Unit 4

Sriram S

PES1UG24CS469

4-H

Screenshots:

1A:

```
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$
```

1B:

```
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$
```

2A:

```
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$
```

2B:

```
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ find .pes/objects -type f
.pes/objects/27/0eceb8f52c98ab781645bbd8701376a7347dcd68afe6feb899a3abc46e15a3
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ xxd .pes/objects/27/0eceb8f52c98ab781645bbd8701376a7347dcd68afe6feb899a3abc46e15a3 | head -20
00000000: 7472 6565 2034 3800 3130 3036 3434 2066  tree 48.100644 f
00000010: 696c 652e 7478 7400 0000 0000 0000 0000  ile.txt.........
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000030: 0000 0000 0000 0000                      ........
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$
```

3A:

```
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ ./pes add file1.txt file2.txt
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.c
  untracked:  test_objects.c
  untracked:  tree.c
  untracked:  test_manual.c
  untracked:  test_manual
  untracked:  test_tree.c
  untracked:  Makefile
  untracked:  index.c
  untracked:  commit.h
  untracked:  commit.c
  untracked:  .gitignore
  untracked:  test_sequence.sh
  untracked:  pes.h
  untracked:  index.h
  untracked:  tree.h
  untracked:  object.c
  untracked:  README.md
  untracked:  test_tree

ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$
```

3B:

```
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776448352 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776448352 6 file2.txt
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$
```

4A:

```
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ ./pes log
commit e5d1b913ccb0bc48d46870a45098ce37c7bfd229d29abf74df02f22cd3628a4a
Author: PES User <pes@localhost>
Date:   1776448804

    Add farewell

commit ce35d0dd4e07d98510ca699585bb26f33aa22102ee402cab5ec599a8223e9356
Author: PES User <pes@localhost>
Date:   1776448799

    Add worldell

commit 8dbe41a79c8015d2b6c3895d106cc427ce9f1d4c42f7bd2a0ec08747596e9842
Author: PES User <pes@localhost>
Date:   1776448783

    Initial committial commiQ

ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$
```

4B:

```
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/48/1407f7237f7bd749dca31e2d8422bafe211741d5556f007e0faf821ffc7179
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/8d/be41a79c8015d2b6c3895d106cc427ce9f1d4c42f7bd2a0ec08747596e9842
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/ce/35d0dd4e07d98510ca699585bb26f33aa22102ee402cab5ec599a8223e9356
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e5/d1b913ccb0bc48d46870a45098ce37c7bfd229d29abf74df02f22cd3628a4a
.pes/objects/f3/97707e14d1cabf1c29a72d833bffb94d4a474ecc48789d6496742ae6541034
.pes/refs/heads/main
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$
```

4C:

```
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
e5d1b913ccb0bc48d46870a45098ce37c7bfd229d29abf74df02f22cd3628a4a
ref: refs/heads/main
ubuntu24.04@Ubuntu24:~/PES1UG24CS469-pes-vcs$
```

Final:

```
[illegible] make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: ed1b10315dcb... Initial commit

Log after first commit:
commit ed1b10315dcbcdcc289174737246ec6340fa630514edfee3a84f3b40d596650e
Author: PES User <pes@localhost>
Date:   1776448867

    Initial commit


--- Second Commit ---
Committed: e90c53bcddd9... Update file.txt

--- Third Commit ---
Committed: 0b236f099d0d... Add farewell

--- Full History ---
commit 0b236f099d0db40410006915253f919e708f647fe0884207e66a7f05aa3dc491
Author: PES User <pes@localhost>
Date:   1776448867

    Add farewell

commit e90c53bcddd9b31679d030312c07e945ff6309b00746518cfd07c30993da2590
Author: PES User <pes@localhost>
Date:   1776448867

    Update file.txt

commit ed1b10315dcbcdcc289174737246ec6340fa630514edfee3a84f3b40d596650e
Author: PES User <pes@localhost>
Date:   1776448867

    Initial committial commiQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
0b236f099d0db40410006915253f919e708f647fe0884207e66a7f05aa3dc491

--- Object Store ---
Objects created:
10
.pes/objects/0b/236f099d0db40410006915253f919e708f647fe0884207e66a7f05aa3dc491
.pes/objects/0b/dd9098bd9b9cc5934a610ab65da429b525301147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef08d4c7c7e0d6095797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09035c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5150f9
.pes/objects/db/07a1451ca9544dbf60d709b505377d765efae7adc6b97b75cc9d2b3b3da0ff
.pes/objects/e9/0c53bcddd9b31679d030312c07e945ff6309b00746518cfd07c30993da2590
.pes/objects/ed/1b10315dcbcdcc289174737246ec6340fa630514edfee3a84f3b40d596650e

=== All integration tests completed ===
```

**Phase 5 & 6:**

**Q5.1:** A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Ans:

To implement pes checkout <branch>:

1. **Update HEAD**
    - Modify .pes/HEAD to: ref: refs/heads/<branch>
2. **Read target commit**
    - Open .pes/refs/heads/<branch>
    - Get commit hash
3. **Load commit → tree**
    - Use object_read() to read commit
    - Extract tree hash
4. **Update working directory**
    - Traverse tree
    - Write all files from blobs to disk

**Why this is complex:**

- Must overwrite existing files safely
- Must handle nested directories
- Must avoid losing uncommitted changes

**Q5.2:** When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse.

Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Ans:

For each file in index:

1. Get current file metadata: stat(path)
2. Compare with index:
    - mtime
    - size
3. If mismatch: file is modified → dirty state

**Q5.3:** "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans:

**What happens:**

- New commits are created
- But no branch points to them
- They can be lost

**Recovery methods:**

1. Create a branch: git branch new_branch
2. Or use: git reflog

to find lost commits

**Q6.1:** Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use

to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Ans:

**Algorithm:**

1. Start from all branch heads: .pes/refs/heads/*
2. Traverse:
    - Commit → parent commits
    - Commit → tree
    - Tree → blobs/subtrees
3. Mark all visited hashes
4. Delete unmarked objects

**Data structure:** Hash Set

**Complexity estimate:**

Traversal ≈ all reachable objects
Worst case: ~100k commits + trees + blobs

**Q6.2:** Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Ans:

**Race condition:**

1. Commit process:
    - Writes new object (not yet referenced)
2. GC runs:
    - Doesn’t see reference → deletes object

3. Commit tries to use it → broken state

**How Git avoids this:**

1. Uses **locks**
2. Uses **temporary files**
3. Uses **mark-and-sweep safely**
4. Avoids deleting recent/unreferenced objects immediately